# Teste de Desempenho

**Integrantes:**

- André Coutinho de Macedo Silva;
- André Rodrigues Farias da Silva;
- Fernanda Maria de Souza.

## Contexto

Os testes de desempenho realizados para esta atividade são referentes a um *endpoint* gerado em uma rede local como uma requisição HTTP do método POST, simulando o cadastro de novos usuários no sistema de gerenciamento de saúde.

## Ambiente de Testes

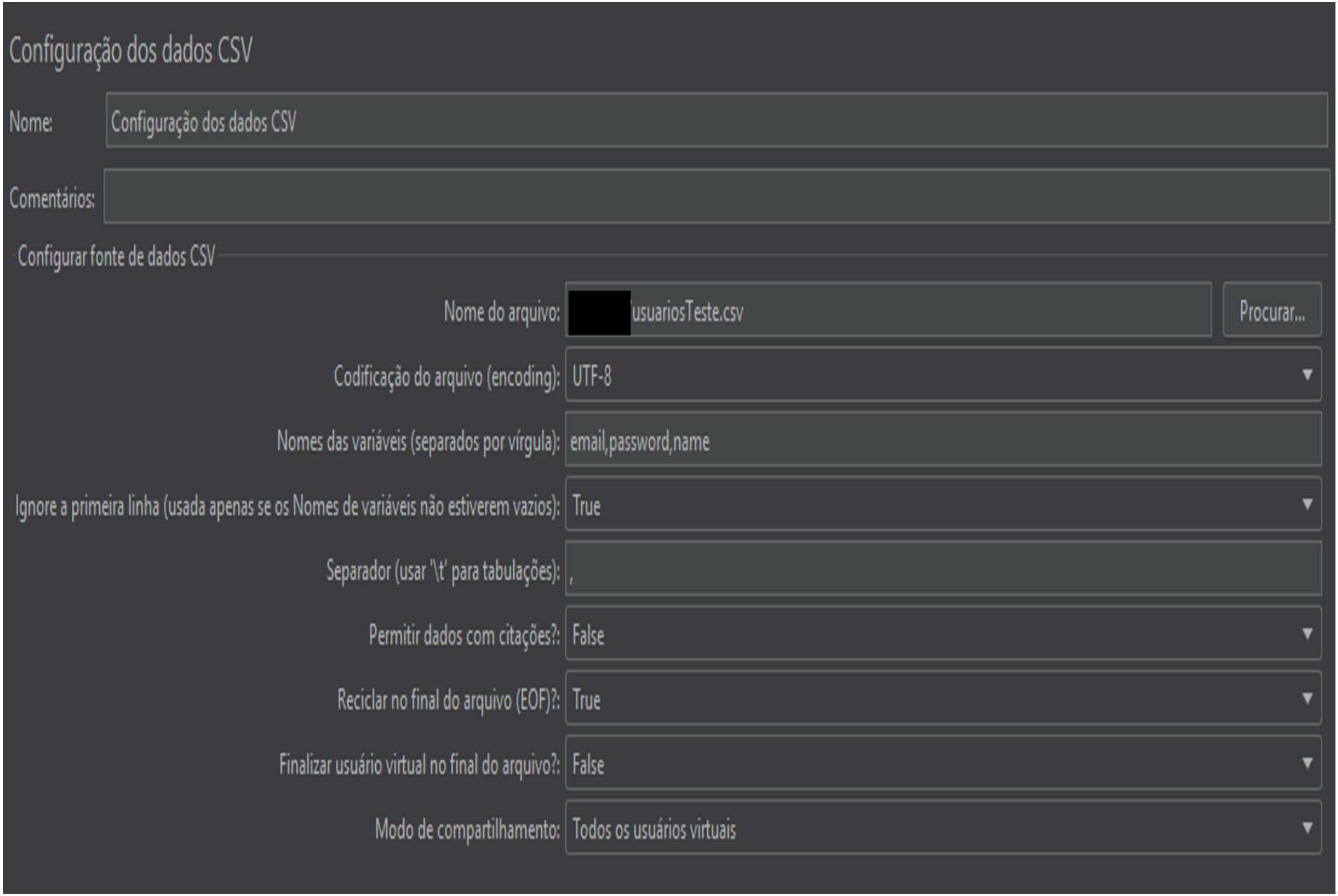
Os testes foram realizados usando o *software* Apache JMeter em um computador pessoal com um sistema operacional de 64 bits na edição Windows 11 Home na versão 25H2. Os grupos de usuário foram organizados com 10 *threads* de usuários, com tempo de inicialização em 10 segundos e um contador de Iteração em 5 loops.

O *back-end* do projeto possui uma regra em que não podem haver usuários cadastrados com endereços de e-mail repetidos. Por esse motivo o ambiente de testes também precisou contar com um CSV contendo uma lista de usuários teste, cada com um com um nome, e-mail e senha diferentes. Dessa forma, o testes de desempenho considera 10 usuários diferentes realizando o cadastro. O CSV cadastrado no JMeter contém os seguintes dados de usuário:

| email | password | name |
|---|---|---|
| test01@mail.com | Test01@1234567 | Primeiro Teste |

| email | password | name |
|---|---|---|
| test02@mail.com | Test02@1234567 | Segundo Teste |
| test03@mail.com | Test03@1234567 | Terceiro Teste |
| test04@mail.com | Test04@1234567 | Quarto Teste |
| test05@mail.com | Test05@1234567 | Quinto Teste |
| test06@mail.com | Test06@1234567 | Sexto Teste |
| test07@mail.com | Test07@1234567 | Setimo Teste |
| test08@mail.com | Test08@1234567 | Oitavo Teste |
| test09@mail.com | Test09@1234567 | Nono Teste |
| test10@mail.com | Test10@1234567 | Decimo Teste |

O CSV foi configurado conforme mostra a figura abaixo:


Configuração dos dados CSV
Nome:
Configuração dos dados CSV
Comentários:
Configurar fonte de dados CSV
Nome do arquivo:
usuariosTeste.csv
Procurar...
Codificação do arquivo (encoding):
UTF-8
Nomes das variáveis (separados por vírgula):
email,password,name
Ignore a primeira linha (usada apenas se os Nomes de variáveis não estiverem vazios):
True
Separador (usar '\t' para tabulações):
,
Permitir dados com citações?:
False
Reciclar no final do arquivo (EOF)?:
True
Finalizar usuário virtual no final do arquivo?:
False
Modo de compartilhamento:
Todos os usuários virtuais

A requisição HTTP do teste de desempenho usa o protocolo http, o servidor `localhost` e a porta `8080`. A requisição é do tipo `POST` e o caminho é `/user/signup`. O corpo dos dados para a requisição responde aos parâmetros do CSV com o tipo de conteúdo no valor de aplicação JSON. A requisição toma a forma abaixo:

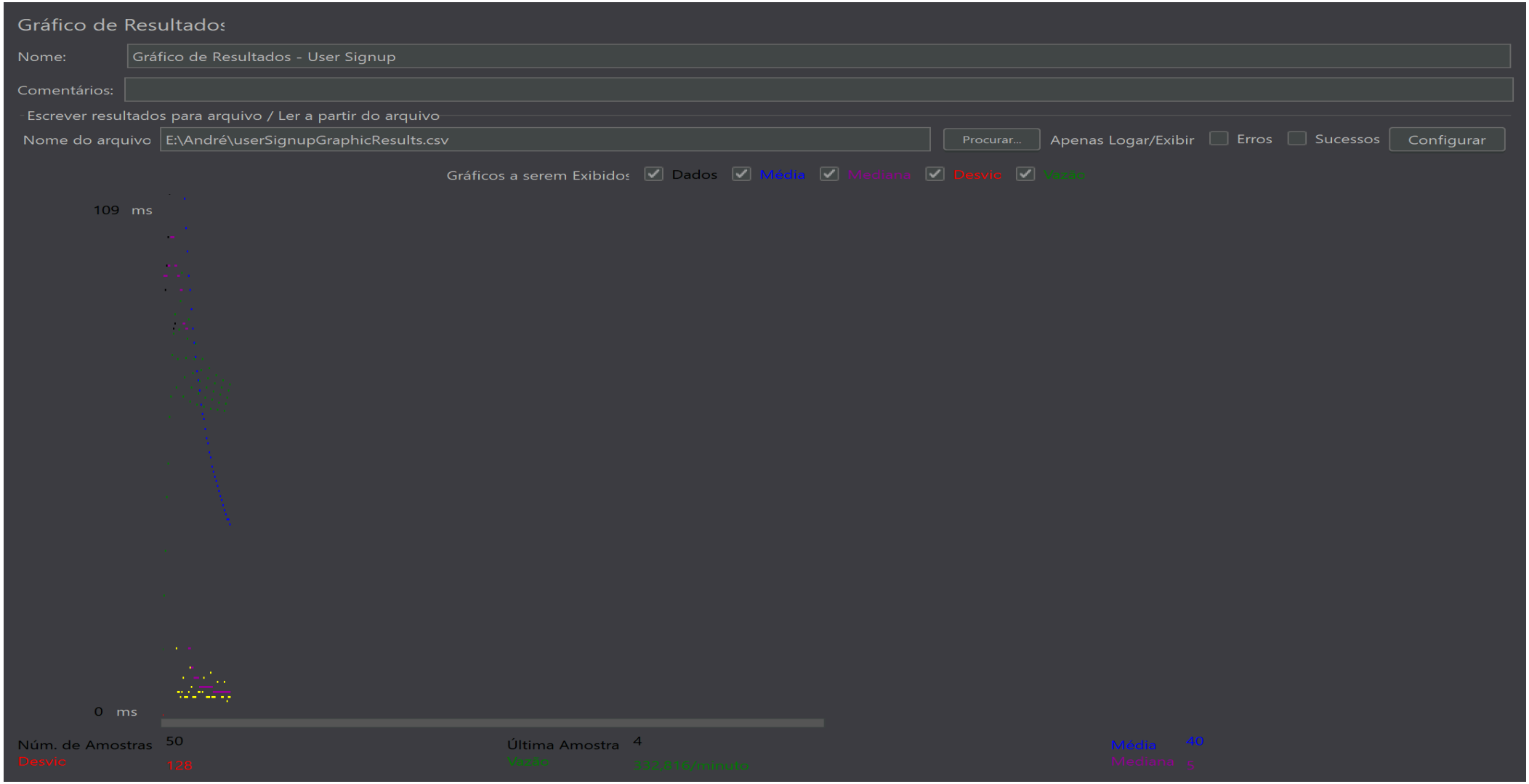
```
{
  "email": "${email}",
  "password": "${password}",
  "name": "${name}"
}
```

# Resultados

## Sumário

| Rótulo | # Amostras | Média | Mín. | Máx. | Desvio Padrão | % de Erro | Vazão | KB/s | Sent KB/sec | Média de Bytes |
|---|---|---|---|---|---|---|---|---|---|---|
| Requisição HTTP - User Signup | 50 | 40 | 3 | 901 | 128,39 | 80,000% | 5,54693 | 2,87 | 1,49 | 529,0 |
| TOTAL | 50 | 40 | 3 | 901 | 128,39 | 80,000% | 5,54693 | 2,87 | 1,49 | 529,0 |

## Gráfico




## Ponto de Atenção

Percebemos que usando a regra do e-mail único, o teste de desempenho só é efetuado no primeiro loop, já que nos 4 seguintes, ele fará a requisição HTTP com 10 usuários já cadastrados. Por esse motivo temos 10 amostras com código de resposta `201` e 40 com `409` negando a aplicação.

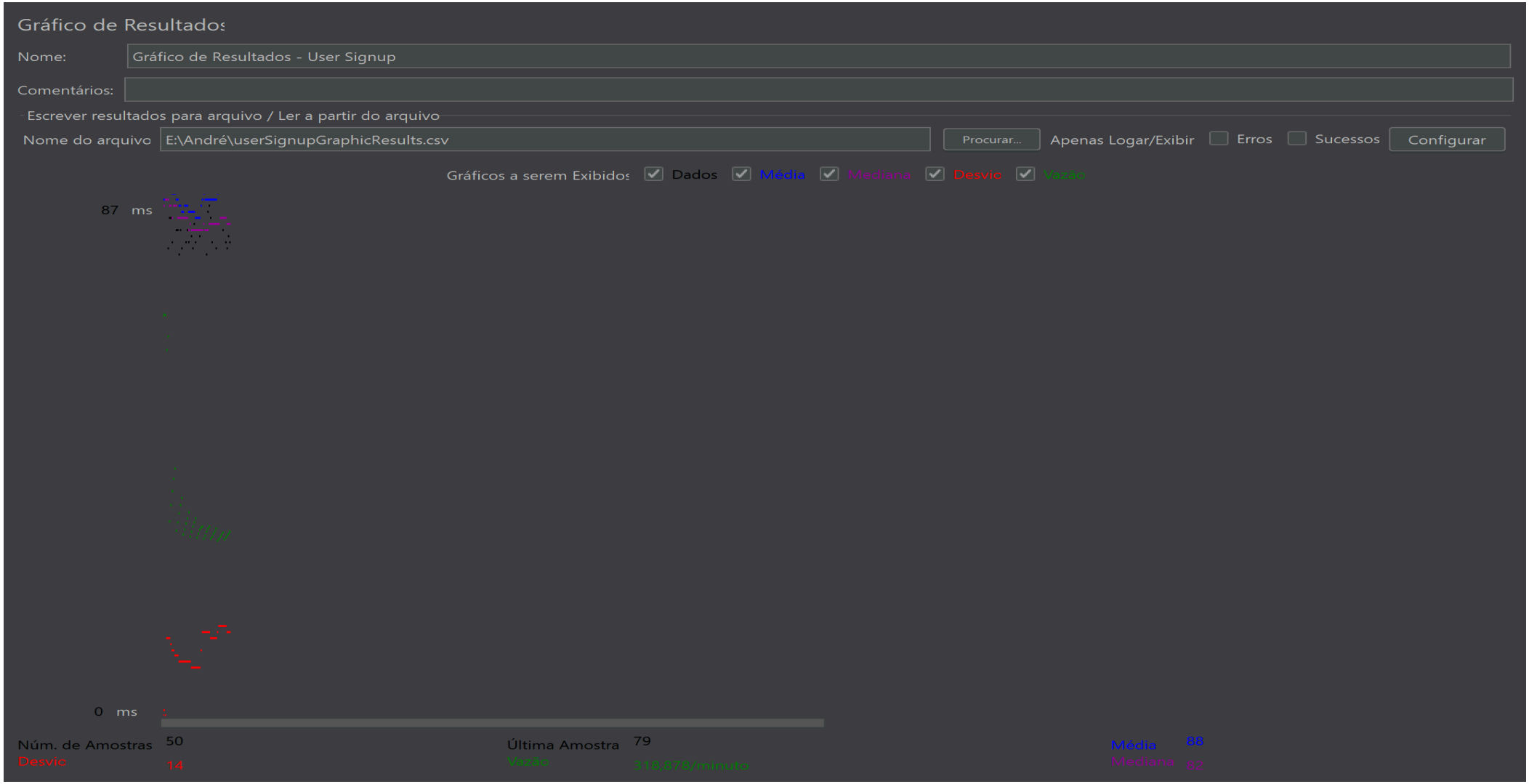
Um outro teste foi preparado com as mesmas configurações no JMeter, mas agora sem a regra do e-mail único. Eis os resultados.

# Resultados 2

## Sumário

| Rótulo | # Amostras | Média | Mín. | Máx. | Desvio Padrão | % de Erro | Vazão | KB/s | Sent KB/sec | Média de Bytes |
|---|---|---|---|---|---|---|---|---|---|---|
| Requisição HTTP - User Signup | 50 | 88 | 77 | 139 | 14,75 | 0,000% | 5,31463 | 3,76 | 1,42 | 723,9 |
| TOTAL | 50 | 88 | 77 | 139 | 14,75 | 0,000% | 5,31463 | 3,76 | 1,42 | 723,9 |

## Gráfico

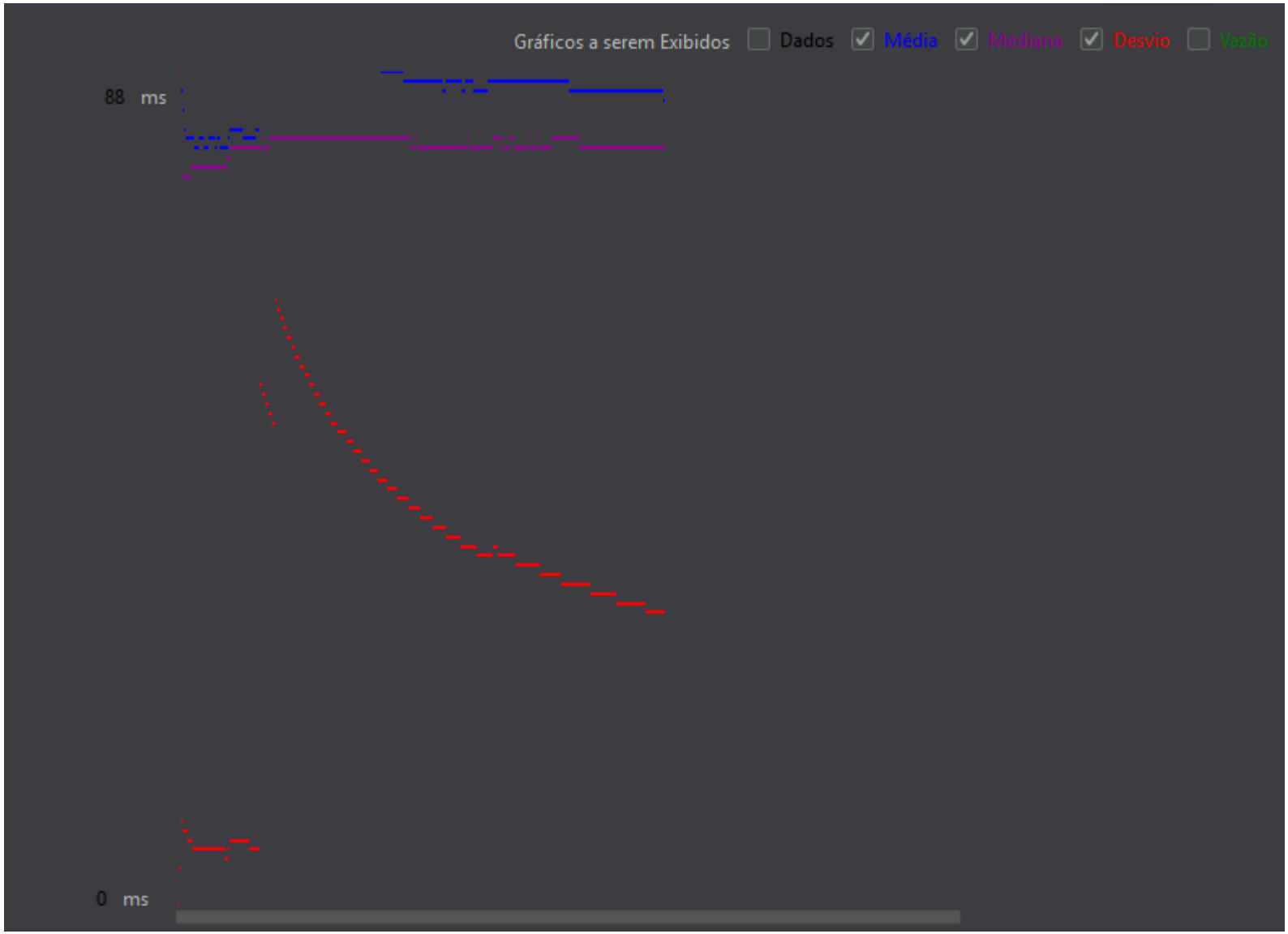



## Relatório da Atividade

O tempo médio de resposta no teste configurado corretamente foi de 88ms, não havendo falhas e com uma taxa de 1,42 kilobytes por segundo. Observamos que, comparado ao primeiro teste feito com a regra do e-mail único, o desvio padrão foi bem mais baixo. Isso é explicado não somente pelas amostras com erro que tinham uma resposta em latência significativamente menor, mas pelo fato do banco precisar ter sido sempre

reiniciado para alocar usuários com e-mails ainda não cadastrados. Sem a regra do e-mail único, o banco pôde permanecer aberto, diminuindo a latência de novos usuários e diminuindo a diferença de latência entre as amostras.

Podemos demonstrar isso reiniciando o banco e fazendo 5 sessões de entrada com 10 usuários simultâneos uma seguida da outra, todas sem a regra do e-mail único e inicializadas na mesma instância de abertura da aplicação.

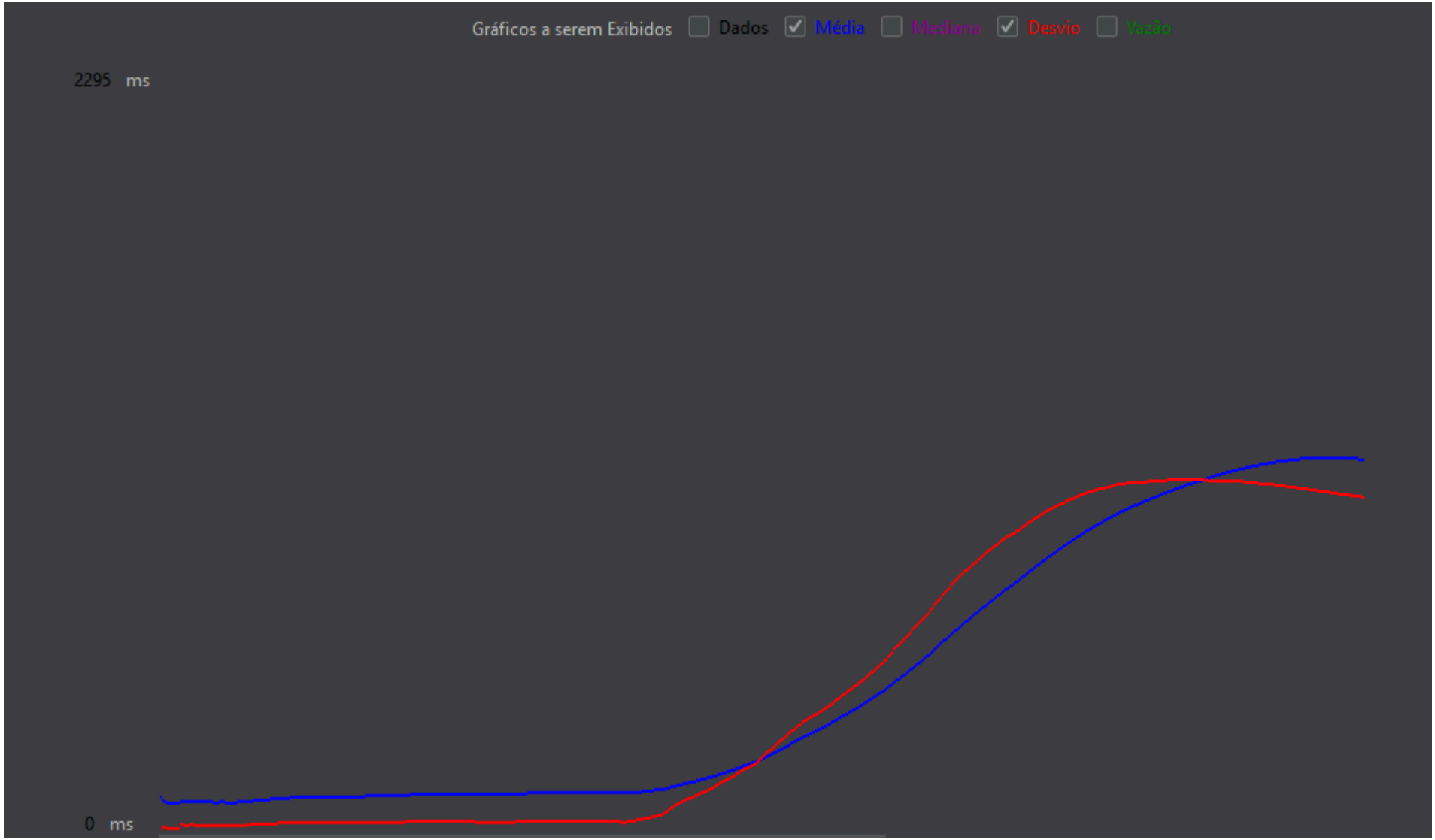


A partir do gráfico percebemos que o desvio padrão diminuiu enquanto alcançava uma estabilidade, mesmo com a mediana e a média já sendo estáveis desde o início. O que indica que, apesar da média de desempenho permanecer estável, mais e mais inicializações fazem com que a latência de diferentes usuários tenham um valor cada vez mais próximo e estável.

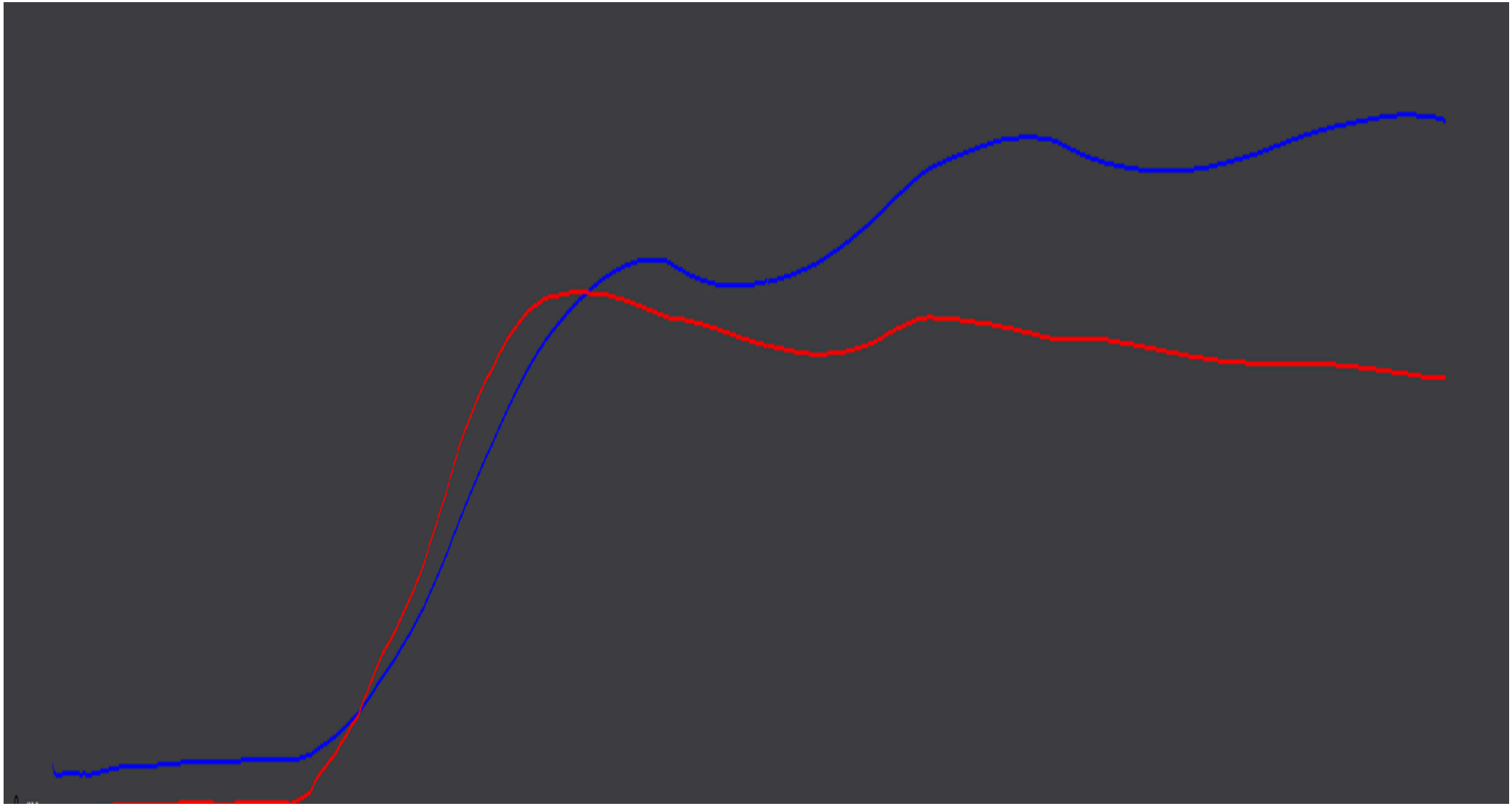
Diante disso, vejamos o que acontece quando, ainda na mesma inicialização do banco, aumentamos o número de usuários por teste de desempenho. Os próximos resultados representam os resultados unidos de três testes de desempenho consecutivos: um com 10 usuários simultâneos, outro com 50 e um último com 100:

| Rótulo | # Amostras | Média | Mín. | Máx. | Desvio Padrão | % de Erro | Vazão | KB/s | Sent KB/sec | Média de Bytes |
|---|---|---|---|---|---|---|---|---|---|---|
| User Signup - 10 usuários | 50 | 83 | 75 | 152 | 14,02 | 0,000% | 5,33333 | 3,78 | 1,43 | 726,1 |
| User Signup - 50 usuários | 250 | 118 | 78 | 203 | 25,59 | 0,000% | 24,41406 | 17,31 | 6,54 | 726,0 |
| User Signup - 100 usuários | 500 | 1693 | 96 | 3026 | 797,09 | 0,000% | 27,17391 | 19,27 | 7,28 | 726,0 |
| TOTAL | 800 | 1100 | 75 | 3026 | 991,55 | 0,000% | 4,58314 | 3,25 | 1,23 | 726,0 |

Podemos observar que quanto mais usuários simultâneos realizam o cadastro, maior fica a latência máxima, a vazão dos dados e o desvio padrão. Apesar disso, o aumento de latência máxima não é tão expressivo quanto o de latência mínima, o que pode indicar uma tendência à estabilidade de latência e um desvio padrão mais baixo, mesmo com mais usuários sendo cadastrados. Observemos o gráfico:




Tanto o desvio padrão como a média apresentam comportamentos similares conforme o número de usuários simultâneos cadastrados. Eles sobem, mas ainda com uma certa "gravidade" que o tendenciam a diminuir mesmo distantes dos testes anteriores. Se continuarmos a cadastrar 100 usuários simultâneos, o gráfico continua o comportamento, apresentado o resultado abaixo:



Assim, o desvio padrão busca uma estabilidade, diminuindo a cada teste. Enquanto isso a média de latência sempre sobe, mas ainda com picos baixos que retardam o aumento desenfreado.

## Conclusão

Precisamos nos atentar a quantos usuários usam nosso sistema ao mesmo tempo. Quanto mais usuários se cadastram simultaneamente, maior a chance de um usuário ter uma latência mais alta do que a maioria. Outro cuidado que devemos ter é o período de inicialização do banco. Sempre que quisermos fazer atualizações de sistema, ou qualquer procedimento que peça a reinicialização do banco, estaremos aumentando as chances de que um usuário tenha uma latência maior para a realização do cadastro. Ainda assim, com os testes efetuados podemos ter mais segurança com relação à conformidade do desempenho, tendo ciência de que mesmo com muitos usuários, o sistema tende a se adaptar e encontrar uma estabilidade latência que diminuirá a discrepância de desempenho entre um usuário e outro.